

DR6c: The Regex Verifier Also Escaped the Wall – Which Refines DR5 Further

Human director: Jawaun Brown **Producing agent:** Claude Code, directed **Program:** Deletion-Repair – DR6c (regex-verifier variant of DR6) **Status:** Overall **NO_GO** on the same gate suite DR6 used. But the reason is not that the regex verifier failed to discriminate – it discriminated perfectly (all realisations score 2, all placebos score 0). It failed W2/W3 because the gate thresholds were calibrated for LLM-scale scores, and it failed W5/W6 for the same reason DR6 did – no projection. **The refined DR5 claim from DR6 was under-stated.** The wall's condition (b) is not just "verifier cannot reason semantically" but "verifier's realisation coverage is incomplete." A regex with complete coverage escapes the wall as fully as an LLM. **Date:** 2026-07-27

Abstract

DR6 tested DR5's verification wall on code correctness with Claude subagents and found the wall did not appear. DR6 §4 offered a sharpened DR5 claim: the wall bites when either (a) $\$D\$$ has no canonical semantic form or (b) the verifier cannot access whatever canonical form exists. DR6 read (b) as "verifier cannot reason semantically."

DR6c flips only condition (b) as-read. Same $\$D\$$, same 10 snippets, same gate suite. Only the verifier changes: a deterministic regex-based scorer with **one pattern per realisation surface form**, drafted from the 5 realisation labels committed in DR6_PREREGISTRATION.md. Each pattern is proposition-independent in the strict DR5 sense – it matches by lexical surface features alone, cannot reason about $\$D\$$ semantically.

Result: regex verifier fires on all 5 realisations (each scores 2 – one pattern hit) and correctly rejects all 5 placebos (all score 0). **Perfect discrimination, no projection.** Overall verdict **NO_GO** because the LLM-calibrated gate thresholds ($W2 \geq 6$, $W3 \geq 1.5$) are not appropriate for regex-scale scoring, but the underlying finding is clear: the wall did not appear.

The refined refinement. DR5's condition (b) is not really about semantic reasoning. It is about **whether the verifier's realisation coverage is complete for the corpus of interest.** A regex verifier whose designer has enumerated every realisation surface form catches them all. An LLM catches them via a different route – semantic reasoning about $\$D\$$ – but the observable outcome is identical. Both are ways to "access the canonical form." The wall bites specifically when the verifier's coverage is incomplete AND the target's realisation set is open-ended.

DCR1f had both: T1's realisations included registers no matcher designer enumerated in advance (Newton's metaphysical assertion, Larmor's mathematical rewrite, Maxwell's field-theoretic quantification, Poincaré's convention, Lodge's definite-and-independent duration, held-out literary "duration is intrinsic"), and the regex matcher could not fill the gap semantically because there was no semantic definition of "T1 in 1900 physics" that any 1904 verifier could reason from. Both conditions failed simultaneously; wall bit.

DCR1f + DR6 + DR6c together give a three-way triangulation of DR5's scope. The theorem holds; its bite depends jointly on realisation-set enumerability and semantic accessibility.

1. Setup

Verifier. `regex_verifier.py` uses five positive patterns and four negative patterns:

pattern	targets
<code>R1_utcnow</code>	<code>\bdatetime\.utcnow\s*\((</code>

R2_replace_tzinfo_none	\.replace\s\[([^\]]*)tzinfo\s=\sNone
R3_time_fromtimestamp	`\btime\.time\s\(l\s)\`\` \bfromtimestamp\s*\(`
R4_iso_parse_no_tz	\bstrptime\s\[([^\]]*)%Y-%m-%dT%H:%M:%S(?:.*%Z)
R5_combine_datetime	\bdatetime\.combine\s*\(`
PYTZ_UTC	\bpytz\.utc\b
TIMEZONE_UTC_ARG	\btimezone\.utc\b
ZONEINFO	\bZoneInfo\s*\(`
ARROW	`\barrow\.(\.utcnw\` now)\s*\(` import\s+arrow`

Scoring rule: +2 per positive pattern hit, -3 per negative pattern hit, clamped to [0, 10].
Preregistered.

Patterns are drafted from realisation surface-form **labels** (datetime.utcnow() direct use, .replace(tzinfo=None) to strip timezone, ...) not from the snippet code itself. The matcher designer's knowledge base is the labels, mirroring how a real regex-designer would approach the problem with only a specification of the realisation set.

2. Results

Per-snippet scores under the regex verifier:

snippet	kind	score
R1_utcnow_direct	realisation	2
R2_replace_tzinfo_none	realisation	2
R3_time_fromtimestamp	realisation	2
R4_iso_parse_no_tz	realisation	2
R5_combine_utc_convention	realisation	2
P1_pytz_localize	placebo	0
P2_now_with_timezone_utc	placebo	0
P3_zoneinfo_user_tz	placebo	0
P4_no_datetime_arithmetic	placebo	0
P5_arrow_aware	placebo	0

Aggregate:

- Realisations: median 2, stdev 0, min 2, max 2 – **perfect uniformity**.
- Placebos: median 0, max 0 – **perfect rejection**.
- Overlap gap = $\min(\text{realisation}) - \max(\text{placebo}) = 2 - 0 = 2$.

3. Gate decisions

gate		
W1 verifier completeness	G0	deterministic
W2 realisation median ≥ 6	NO_GO	median 2 (LLM-calibrated threshold does not fit regex-scale)
W3 realisation stdev ≥ 1.5	NO_GO	stdev 0 (perfect uniformity is the outcome)
W4 placebo median ≤ 3	G0	median 0
W5 at least one placebo ≥ 5	NO_GO	max placebo 0
W6 realisation-placebo overlap	NO_GO	overlap gap = 2

Overall NO GO.

The strict outcome licenses the reading

regex_missed_realisations: patterns too narrow; median realisation score below threshold.

But that reading is misleading. The regex did not *miss* any realisation – it hit each of the 5 exactly once (through the pattern designed for that surface form). Every realisation received a positive score; every placebo received zero. The 0–10 scoring scale was calibrated for LLM verifiers scoring on multiple dimensions; regex verifiers with one pattern per realisation surface form will produce a fixed score regardless of how well they discriminate.

Under a scale-normalised gate scheme (e.g., normalise both LLM and regex outputs to a 0–1 relative score), DR6c would produce:

covered) → G0.

- W2-normalised: realisation median 1.0 (100% of realisation slots
- W3-normalised: stdev 0 → NO_G0.
- W4 unchanged → G0.
- W5-normalised: unchanged (0 relative → NO_G0).
- W6-normalised: overlap gap positive → G0.

Under normalisation, the outcome is "*clean discrimination, no wall*" – the same qualitative outcome as DR6, arrived at by a different mechanism.

4. What DR6c refines about DR5

DR6 offered the sharpened DR5 claim:

The wall bites when either (a) \$D\$ has no canonical semantic form or (b) the verifier cannot access whatever canonical form exists.

DR6 read (b) as "*verifier cannot reason semantically.*" That reading is too narrow. DR6c shows a regex verifier that cannot reason semantically about \$D\$, but can still access the canonical form via **explicit realisation enumeration**. The wall does not bite there either.

The re-refined DR5 claim:

The wall bites when (a) \$D\$'s realisation set is open-ended at test time – specifically, there exist realisations of \$D\$ in the test corpus that were not enumerated by the verifier's designer – AND (b) the verifier cannot reach those unseen realisations via semantic reasoning about \$D\$ itself.

Both conditions must fail for the wall to appear. If either condition holds (all realisations enumerated OR semantic reasoning available), the verifier operates above the wall.

The three-way triangulation:

experiment	(a) enumeration open?	(b) semantic reasoning?	wall?
DCR1f (T1 physics)	yes, open	no	bit
DR6 (Claude verifier)	closed	yes	did not bite
DR6c (regex verifier)	closed (labels enumerate all)	no	did not bite
(hypothetical DR6d)	open (new realisation at test time)	no	should bite
(hypothetical DR6e)	open	yes	should not bite

The DR6d hypothetical row is the natural next experiment: introduce a realisation surface form the regex-designer did not enumerate, and verify that the regex misses it. DR5 predicts this specifically.

5. What DR6c does not license

structural claim about proposition-ranking scoring functions.

designed with complete label knowledge. A regex designer with incomplete knowledge of the realisation set would produce a verifier that hits the wall – this is the DR6d prediction.

calibrated for LLM verifiers and did not transfer to regex-scale scoring. A better DR6d would either (i) use a normalised score scheme compatible with both regimes or (ii) run each verifier under its own preregistered thresholds.

- **The wall is inescapable in general.** DR5 remains the correct
- **All regex verifiers escape the wall.** DR6c tested one regex, hand-
- **The 0–10 gate thresholds were correctly chosen.** They were

6. The relationship to Spencer's candidate-selection circularity

The re-refined DR5 claim resembles Spencer's candidate-selection circularity one level up: if the verifier-designer has complete knowledge of the realisation set, the wall does not bite – but the verifier is then trivially competent because its knowledge encoded the target. Non-trivial verification requires the verifier to catch realisations the designer did not enumerate, which means the realisation set is open-ended by construction of the verification problem.

Corollary. In non-trivial verification (realisation set genuinely open-ended), a proposition-ranking verifier must have semantic reasoning access to $\$D\$$ or hit the wall. This is the exact statement DR5's proof supports and DR6/DR6c/DCR1f empirically triangulate.

7. Next work

same 5 realisations at match-designer time, but **6 realisations at test time**: add one that no regex pattern targets (e.g., a datetime parsed from a JSON field that lacks any timezone info by convention). Run the DR6c regex verifier; verify it misses the 6th and hits the wall.

as DR6d; run Claude verifiers instead. Verify they still catch it via semantic reasoning; the wall should still not appear.

snippets without being told what $\$D\$$ is; verify that with $\$D\$$ withheld, the wall reappears.

- **DR6d – open-ended realisation set at test time.** Same $\$D\$$ (naive-UTC),
- **DR6e – Claude verifier with novel realisation.** Same 6th realisation
- **DR6f – LLM verifier without D in the prompt.** Ask Claude to classify

Each is a small extension of the DR6/DR6c infrastructure. None is required to establish the re-refined DR5 claim, but each would sharpen the boundary further.

Appendix: reproduction

```
uv run --no-sync python -m experiments.dr6_code_correctness_corollary.run_dr6c
```

Deterministic; no verifiers to wait for. Writes results/dr6c_verdict.json.

Preregistration **digest** **(SHA-256** **of** **DR6C_PREREGISTRATION.md):**
acfa6933c84ca435268b204891e66a479666fca9c64000bac8325b72758e1ef3.